P.Vishwa Nandan(192473015)

TASK B2 – ENGINEER AN ADVERSARIAL INPUT (5 MARKS)

1. Aim

To test the security and reliability of Java database and file-handling programs using deliberately constructed adversarial inputs.

The task implements unsafe and secure login checks using JDBC, tests SQL injection, and verifies how the bus-pass file store handles damaged records.

2. Problem Description

This task contains two experiments.

Experiment 1 – SQL Injection

The original login program directly joins user input into an SQL query. An attacker may enter specially crafted input to bypass authentication.

The application must demonstrate the difference between:

- Unsafe login using Statement.
- Secure login using PreparedStatement.

Experiment 2 – Damaged Bus-Pass File

The bus-pass file may contain:

- Records with the wrong number of fields.
- Non-numeric amounts.
- Empty lines.

The application must identify or skip these records without stopping the entire search.

3. Experiment 1 – Unsafe and Secure Login

3.1 Local Test Database Setup

Create a separate database exclusively for this assignment.

Do not perform SQL injection testing on a real college, university, or production database.

SQL Program – security_test.sql

```
CREATE DATABASE IF NOT EXISTS assignment_security;


USE assignment_security;
```

```
CREATE TABLE IF NOT EXISTS users (

    id VARCHAR(50) PRIMARY KEY,

    pwd VARCHAR(100) NOT NULL
);

INSERT INTO users (id, pwd)
VALUES ('admin', 'admin123')
ON DUPLICATE KEY UPDATE pwd = 'admin123';

-- Verify the test account
SELECT * FROM users;
```

Explanation:

1. The database assignment_security stores the test login information.
2. The users table contains an ID and password.
3. A sample account named admin is inserted.
4. This account is used to test normal and malicious login attempts.

The plaintext password is used only to demonstrate the assignment's original vulnerable query. A real application must store salted password hashes, not plaintext passwords.

3.2 Understand the Unsafe Login Query

The original program uses:

```
String sql =
    "SELECT * FROM users WHERE id='" + id
    + "' AND pwd='" + pwd + "'";
```

The problem is that user input becomes part of the SQL command itself.

A malicious input can change the meaning of the query.

3.3 Construct an Adversarial Input

Enter the following values:

User ID:

' OR '1'='1' --

Password:

wrongpassword

The generated SQL becomes:

SELECT * FROM users

WHERE id=''

OR '1'='1' -- ' AND pwd='wrongpassword'

Why Does This Query Accept the Login?

1. The condition '1'='1' is always true.
2. The OR operator makes the WHERE condition true for rows in the table.
3. The -- characters begin an SQL comment in MySQL.
4. The remaining password condition is ignored.
5. Since the test database contains a user record, the query returns a result.

Therefore, the unsafe login may accept an invalid username and password.

3.4 Java Program – UnsafeAndSafeLogin.java

3.5 Program Explanation

Step 1 – Database Connection

The getConnection() method establishes a connection to the local MySQL test database.

Step 2 – Unsafe Login

The unsafeLogin() method joins the supplied username and password directly into the SQL query.

It executes the query using Statement.

Step 3 – Secure Login

The safeLogin() method uses PreparedStatement with two parameter placeholders.

The entered username and password are supplied using setString().

Step 4 – Execute Both Methods

The main method supplies the same malicious username and invalid password to both login methods.

Step 5 – Compare Results

The unsafe version may accept the login because the injected SQL condition changes the query logic.

The secure version treats the attack string as an ordinary username and rejects it.

3.6 Expected Output

Unsafe SQL:

SELECT * FROM users WHERE id='' OR '1'='1' -- ' AND pwd='wrongpassword'

Unsafe login result: ACCEPTED

PreparedStatement result: REJECTED

The output assumes that the local test database contains at least one user record and the MySQL server accepts the supplied test query.

3.7 Comparison

Feature	**Unsafe Statement**	**PreparedStatement**
User input	Joined directly into SQL	Passed as parameters
SQL injection	Vulnerable	Prevents parameter-based SQL injection
Query execution	Dynamic SQL string	Parameterized SQL
Security	Weak	Stronger
Recommended	No	Yes

4. Experiment 2 – Damaged Bus-Pass File

4.1 Aim

To create a damaged bus-pass file containing three types of invalid records and verify whether the PassStore implementation correctly skips them.

The three damaged conditions are:

1. Wrong number of fields.
2. Non-numeric amount.
3. Empty line.

The file also contains valid records to verify that searching continues after encountering damaged data.

4.2 Construct the Damaged File

Create a file named:

damaged-passes.csv

Its contents are:

P3001,192311001,Arun,12,4500.0

P3002,192311002,Divya,12

P3003,192311003,Ravi,12,ABC

P3004,192311004,Meena,12,4200.0

Explanation of Each Record

Line	Record	Condition	Expected Behaviour
1	P3001,192311001,Arun,12,4500.0	Valid	Accepted
2	P3002,192311002,Divya,12	Only four fields	Skipped
3	P3003,192311003,Ravi,12,ABC	Non-numeric amount	Skipped
4	Empty line	No data	Skipped
5	P3004,192311004,Meena,12,4200.0	Valid	Accepted

4.3 Java Program – PassStoreAdversarialTest.java

4.4 Program Explanation

Step 1 – Create Damaged File

The program creates five lines containing valid and invalid bus-pass records.

Step 2 – Create PassStore Object

PassStore store = new PassStore(file);

This creates the file-store object for the damaged test file.

Step 3 – Search for P3004

The program reads the file line by line.

It skips the damaged P3002 and P3003 records and the empty line.

It finally finds P3004.

Step 4 – Search for P3002

P3002 contains only four fields instead of five.

The PassStore validation rejects it and returns null.

Step 5 – Search for P3003

P3003 contains the amount ABC.

Since the amount should be numeric, the record is rejected.

Step 6 – List Route 12

The listByRoute() method returns only valid records belonging to route 12.

4.5 Expected Output

Damaged bus-pass file created.

Testing PassStore:

Find P3004:

[P3004, 192311004, Meena, 12, 4200.0]

Find P3002:

null

Find P3003:

null

Valid passes on Route 12:

[P3001, 192311001, Arun, 12, 4500.0]

[P3004, 192311004, Meena, 12, 4200.0]

4.6 How Does the File Store Handle Each Adversarial Input?

1. Wrong Number of Fields

The program expects five fields.

P3002 contains only four fields, so the validation method rejects it.

2. Non-Numeric Amount

P3003 contains ABC instead of a valid numeric amount.

The conversion to double fails, and the record is skipped.

3. Empty Line

The CSV parser treats the empty line as an invalid record because it does not contain five fields.

Therefore, the line is skipped.

Important: The current PassStore implementation skips damaged records but does not create a separate damage log. A production system should also record the line number and reason for rejection so that damaged data can be investigated.

5. What Does Each Adversarial Input Reveal?

5.1 SQL Injection Input

The malicious username demonstrates that joining user input directly into an SQL query can change its logical conditions.

It reveals that the original login design does not safely separate commands from data.

5.2 Wrong Number of Fields

The P3002 record demonstrates that a file may contain incomplete or truncated records.

The program must check the expected number of fields before accessing them.

5.3 Non-Numeric Amount

The P3003 record demonstrates that a field may contain the wrong data type.

The program must validate numeric fields before processing them.

5.4 Empty Line

The empty line demonstrates that files may contain blank records due to editing errors or unexpected file modifications.

The application must ignore or report empty lines rather than crashing.

5.5 Overall Observation

Adversarial testing reveals problems that may not appear during normal execution.

It verifies whether the system can reject malicious input, identify invalid records, and continue processing valid data.

6. Alternative Concept and Complexity Analysis

6.1 Alternative – Escaping Quotes Before Building SQL

Instead of using PreparedStatement, one possible approach is to escape single quotes in user input before joining it into an SQL statement.

Example:

```
String safeId = id.replace("'", "''");


String safePassword = pwd.replace("'", "''");


String sql =
   "SELECT * FROM users WHERE id='"
   + safeId
   + "' AND pwd='"
   + safePassword
   + "'";
```

Explanation

This approach replaces every single quote with two single quotes.

For example:

Original input:

admin' OR '1'='1

Escaped input:

admin'' OR ''1''=''1

This can prevent some basic quote-based attacks when the database interprets the escaping as expected.

However, manual escaping is not a reliable replacement for PreparedStatement.

6.2 Alternative – Regular Expression Validation

Another approach is to allow only specific characters in username and password fields.

Example:

```
if (!id.matches("[A-Za-z0-9_]{3,30}")) {


   System.out.println("Invalid username.");
```

```
    return;
}
```

This restricts the username to letters, numbers, and underscores.

However, regular-expression validation alone cannot guarantee SQL security.

It may also reject legitimate values and still fail to protect other input fields or query contexts.

6.3 Why Are These Approaches Weaker?

Manual Quote Escaping:

1. Escaping rules can differ between database systems.
2. Special characters and character encodings require careful handling.
3. Different SQL contexts may require different escaping.
4. Developers may forget to escape one parameter.
5. Manual escaping creates opportunities for security mistakes.

Regular Expression Validation:

1. Legitimate input may be rejected unnecessarily.
2. The allowed character set may change in the future.
3. Not all SQL injection attempts depend only on single quotes.
4. Validation does not separate SQL commands from parameter data.
5. One incorrectly validated field can still create a vulnerability.

PreparedStatement is preferred because it keeps SQL structure separate from user-supplied parameter values.

Input validation is still necessary, but it should be used alongside PreparedStatement, not instead of it.

6.4 Time Complexity Analysis

Let:

- L = Length of the user input.
- Q = Database query execution time.

Manual Quote Escaping

The program examines the input and creates an escaped string.

Time complexity:

O(L)

Regular Expression Validation

The program checks the input against a pattern.

For a simple anchored pattern without complex backtracking:

O(L)

PreparedStatement

The application binds the values to the prepared query.

Parameter handling is proportional to the input length:

O(L)

The database query itself also requires execution time, represented by Q.

The total operation includes both parameter handling and database execution.

6.5 Comparison Table

Feature	**Manual Escaping**	**Regex Validation**	**PreparedStatement**
Input processing	O(L)	O(L) for a simple pattern	O(L) parameter handling
SQL injection protection	Incomplete	Incomplete alone	Strong protection for bound parameters
Query structure separated from data	No	No	Yes
Maintenance	Difficult	Pattern-dependent	Easier
Database compatibility	Escaping rules vary	Pattern must be maintained	Driver handles parameter binding
Recommended	No	As an additional check	Yes

PreparedStatement does not necessarily have a lower time complexity than manual escaping. Its main advantage is correct separation of SQL commands and user data.

7. Testing and Verification

The following test cases verify both experiments.

7.1 Login Security Tests

Test Case	Input	Unsafe Version	PreparedStatement
TC1	admin / admin123	Accepted	Accepted
TC2	admin / wrongpassword	Rejected	Rejected
TC3	invaliduser / password	Rejected	Rejected
TC4	SQL injection username / wrong password	May be accepted	Rejected
TC5	Empty username and password	Rejected	Rejected

The SQL injection test should be executed only against the local assignment database.

7.2 Damaged File Tests

Test Case	Damaged Input	Expected Behaviour
TC1	Four fields instead of five	Record skipped
TC2	Amount contains ABC	Record skipped
TC3	Empty line	Line skipped
TC4	Valid record after damaged records	Record successfully found
TC5	Search for damaged Pass ID	Returns null
TC6	List by route	Only valid matching records returned

8. Final Result

The unsafe login query and secure PreparedStatement implementation were developed and tested using a local MySQL database.

A deliberately crafted SQL injection input demonstrated that the unsafe login query could accept an invalid login, whereas PreparedStatement treated the malicious input as ordinary data and rejected it.

A damaged bus-pass file was also created containing an incorrect field count, a non-numeric amount, and an empty line.

The PassStore implementation successfully skipped the damaged records and continued searching and listing valid records.

Finally, manual escaping and regular-expression validation were compared with PreparedStatement using security, maintainability, and time complexity.

Thus, the task demonstrates secure SQL handling and reliable processing of adversarial file data.